



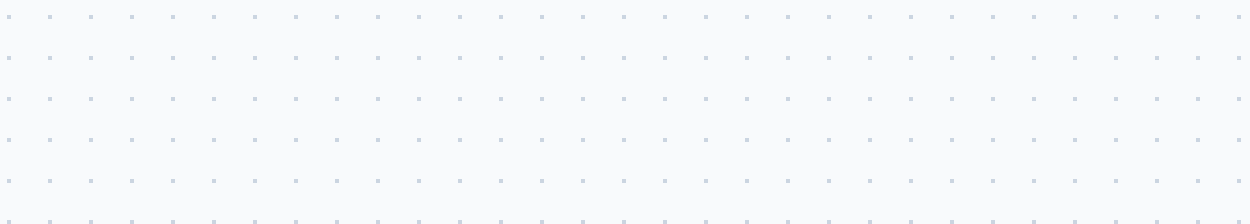
08 — Closure

JAVASCRIPT MECHANICS PRIVACY STATE

THE CORE CONCEPT. A closure occurs when an inner function retains access to the variables of its outer function, even after the outer function has finished running. This securely traps the data in a private persistent state.

THE NUANCE. Normally, when a function finishes executing, its local variables are destroyed by the JavaScript garbage collector. However, if that function returns an inner function that still references those local variables, the engine preserves that specific variable environment (the activation record) in memory. This creates truly private state. The variables are entirely inaccessible from the outside world, forming an impenetrable vault that can only be modified through the returned "privileged" methods.

FRAMEWORK CONTEXT. Svelte's reactivity system relies heavily on closures. It is the foundational JavaScript mechanic that modern framework systems use to encapsulate data and reliably track reactive changes over time. When you write a reactive `$effect()`, Svelte executes this closure to track dependencies, and because the closure holds onto those variables, it can accurately re-evaluate the exact same logic whenever the underlying state changes.



counter.js

```
function createCounter() {  
  let count = 0; // Private state  
  return function() {  
    count++;  
    return count;  
  };  
}  
  
const increment = createCounter();  
console.log(increment()); // 1  
console.log(increment()); // 2
```

LEXICAL ENVIRONMENT

OUTER FUNCTION

createcounter()



- ① **The move.** Closures create private state vaults. By returning a function that references outer variables, you prevent those variables from being garbage collected, giving you precise, controlled access to state.